CSCI 2406: Topic 01

Computer Architecture & Organization Fundamentals

Architecture vs. Organization, ISA vs. Microarchitecture, Stored-Program Concept, and Memory Organization

Computer Organization & Architecture | AArch64 Reference

Course Information & Administration

Instructor Contact

- **Email**: jburns@ada.edu.az
- **Office**: B317
- **Office Hours**: Thursdays 11:00–15:00 (or by appointment)
- *Note*: Avoid Blackboard messages for urgent matters.

Classroom Expectations

- **The Golden Rule**: Once class commences and the instructor speaks, no outside conversations are permitted.
- **Authentic Work**: Emphasize the *why* alongside the *what* and *how*. Do not pass off AI-generated output as your own.

Tools, Emulation, and Assessments

Software & Emulation Stack

- **C & ARM64 Assembly**: Core languages used for architecture programming.
- **QEMU**: Full-system and user-space emulator for AArch64 execution.
- **Compiler Explorer**: Live ARM64 assembly inspection from C source.

Online Programming Resources

- **WebAssembliss**: Browser-based ARM64 Linux assembly environment.
- **CPUlator**: Interactive ARM32 / ARMv7 simulator used where appropriate for comparison.

Reference Platform

Examples use AArch64 as the primary reference. Selected exercises use AArch32 only when a comparison clarifies an architectural difference.

Topic 1 Outline

Topic Goal

Establish the distinction between what software can observe and how hardware implements it, then connect that distinction to stored programs and instruction/data memory organization.

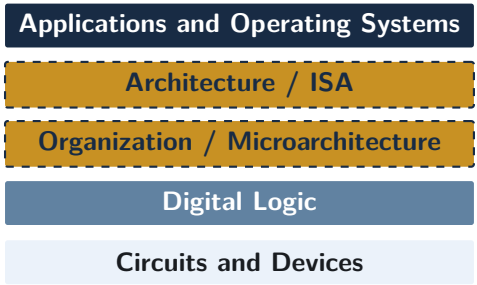
- **Section 1**: Key Abstraction Layers
- **Section 2**: Computer Architecture versus Computer Organization
- **Section 3**: Instruction Set Architecture (ISA) versus Microarchitecture
- **Section 4**: The Stored-Program Concept and Basic Execution
- **Section 5**: von Neumann, Harvard, and Modified Harvard Organization
- **Section 6**: Summary and Self-Test

Section 01

Key Abstraction Layers

Key Levels of Computer Abstraction

Computers manage complexity by separating the system into layers with well-defined interfaces.



Programs, compilers, OS services

Instructions, registers, visible behavior

Datapaths, control, pipelines, caches

ALUs, decoders, register files

Transistors, timing, physical implementation

Today's Focus

The central distinction is between **Architecture / ISA** and **Organization / Microarchitecture**.

Where Software Meets Hardware

Software Side

- Applications
- Compilers
- Operating systems
- Assembly programs

Architecture / ISA Boundary

Defines the machine behavior software may rely on:

- instructions
- visible registers
- state changes
- memory behavior

Hardware Side

- Datapaths
- Control logic
- Pipelines
- Caches
- Execution units

Software targeting the same ISA and compatible software environment can run on processors with very different internal organizations.

Section 02

Computer Architecture vs. Computer Organization

What is Computer Architecture?

Computer Architecture defines the **externally visible capabilities and behavior** of the machine—the **what**.

- The programmer-visible specification of the computer system.
- Establishes what software may request and what architectural state changes occur.
- Provides the stable target used by compilers, operating systems, and assembly programmers.

Architectural Specifications Include

- The **ISA**: instruction encodings, operations, and semantics.
- Programmer-visible state: registers, stack pointer, program counter state, and status information.
- Data types, addressing behavior, memory model, and exception behavior.

What is Computer Organization?

Computer Organization describes the **internal hardware implementation** of the architecture—the **how**.

- The arrangement of operational units and their interconnections.
- Defines how the machine physically realizes each architectural operation.
- Internal details are generally **not part of the programmer-visible architectural contract**.

Organizational Structures Include

- Datapaths, control units, ALUs, register files, and multiplexer routing.
- Pipeline organization, branch prediction, and cache hierarchies.
- Internal interconnects, clocking, memory controllers, and implementation-specific timing.

Architecture vs. Organization: Direct Contrast

Question	Architecture	Organization
Core idea	**What** must the machine do?	**How** is it implemented?
Who sees it?	Programmers, compilers, operating systems	Hardware designers; normally hidden from software
Examples	Instructions, registers, data types, addressing modes	Datapaths, pipelines, caches, control logic
Multiply example	Does the ISA provide a 64-bit multiply instruction?	Dedicated multiplier, Booth recoder, or iterative hardware?
Across CPUs	Meaning must remain compatible	Implementation may differ substantially
Design target	Machine code / assembly interface	Register files, logic, timing, physical implementation

Architectural Stability and Hardware Evolution

Why a Stable Architecture Matters

A stable ISA allows software to retain the same machine-level meaning while processor implementations evolve for performance, power, and cost.

Architectural Stability

- AArch64 defines instructions such as add, `ldr`, and `cbz`.
- Their architectural meaning remains stable across compliant implementations.
- Binary compatibility also depends on the operating environment, ABI, and supported extensions.

Organizational Evolution

- One core may use a modest in-order pipeline.
- Another may use wide out-of-order execution and larger cache hierarchies.
- Both can preserve the same architectural instruction semantics.

Section 03

ISA versus Microarchitecture

The Instruction Set Architecture (ISA)

The **ISA** defines the processor interface visible to machine-language software.

- Instructions and their semantics.
- Programmer-visible registers and status state.
- Instruction encodings and operand forms.
- Memory-access and exception behavior exposed to software.

Architectural Contract: `add x1, x2, x3`

The ISA guarantees the architectural state change:

$$\mathrm{x1} \leftarrow \mathrm{x2} + \mathrm{x3}$$

It defines *what* result must become visible, not the internal wiring or timing used to produce it.

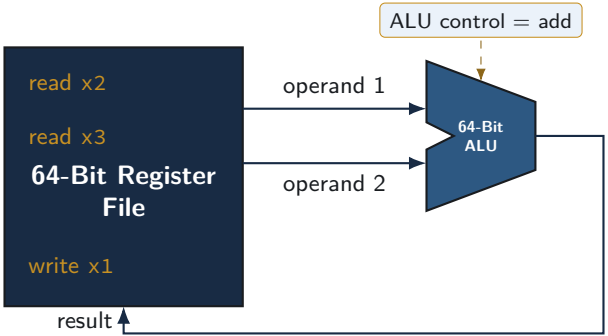
One Possible Microarchitecture for `add x1, x2, x3`

Illustrative Register-Transfer Steps

1. Present `x2` and `x3` as register-file read addresses.
2. Read the two 64-bit operand values from the register file.
3. Configure the ALU to perform addition.
4. Compute: `alu_result <- x2 + x3`.
5. Enable a write to destination register `x1`.
6. Commit the result to the architectural register file.

- **ISA view**: required state change.
- **Microarchitecture view**: control signals, data movement, and execution resources.
- Signal names here are teaching abstractions, not AArch64 architectural state.

Visualizing the Microarchitectural Datapath



Different microarchitectures may perform these internal actions in one long cycle, across several cycles, or through overlapping pipeline stages.

One ISA — Multiple Hardware Realizations

Microarchitecture	Datapath Style	Design Objective
Single-Cycle	Every instruction completes in one long cycle	Simple control; long clock period
Multi-Cycle	Instructions use multiple shorter cycles	Reuse hardware resources
Pipelined	Overlap stages from different instructions	Higher instruction throughput
Superscalar / OoO	Multiple execution paths and dynamic scheduling	High performance at greater complexity

Key Principle

All can implement the same AArch64 instruction semantics while using very different internal organizations.

Section 04

The Stored-Program Concept

The Stored-Program Concept

Stored-Program Definition

Program instructions are represented as binary information and stored in memory, allowing the computer to execute different programs without changing the hardware.

- The processor stores and retrieves program information and runtime data electronically.
- A program counter identifies the next instruction to be fetched.
- Reprogramming means loading a different instruction sequence rather than rewiring the machine.
- The stored-program idea does **not** require instruction and data accesses to use the same physical memory path.

Memory Organization Is a Separate Choice

Von Neumann and Harvard are different ways to organize instruction and data access in a stored-program computer.

Memory as an Addressable Array

- Memory is viewed logically as a sequence of storage locations.
- AArch64 systems are typically **byte-addressed**: each address identifies one byte.
- Larger values span consecutive byte addresses.
- The address identifies *where*; the stored bits are the *payload*.

Address	Byte
`0x1000`	`...`
`0x1001`	`...`
`0x1002`	`...`
`0x1003`	`...`

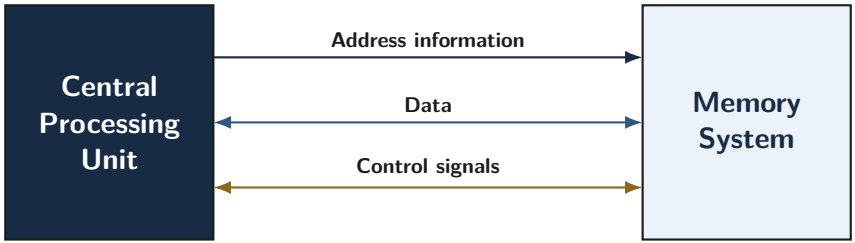
Example

A 32-bit value beginning at `0x1000` occupies addresses `0x1000`–`0x1003`.

What a Memory Access Must Specify

How the CPU Communicates with Memory

Every memory access requires three kinds of information: **where** to access, **what data** to transfer, and **what operation** to perform.



- **Address information**: identifies the memory location.
- **Data**: carries the value being read or written.
- **Control signals**: indicate operations such as read or write and coordinate the transfer.

The traditional "address bus / data bus / control bus" model is a useful abstraction. Modern processors may use more complex interconnects internally.

Context Determines the Meaning of Bits

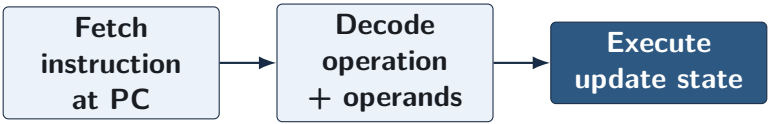
Memory stores bit patterns; their meaning depends on how the processor and software use them.

Access context	**Interpretation of `0x8B020020`**
Instruction fetch	AArch64 instruction add `x0, x1, x2`
Unsigned load	Unsigned 32-bit numerical magnitude
Signed load	Signed 32-bit two's-complement value
Byte access	Four individual 8-bit quantities

Connection to Stored Programs

Instructions can reside in memory because machine instructions are encoded as bit patterns, just like data.

Fetch, Decode, Execute



- **Fetch**: obtain the instruction identified by the current program counter.
- **Decode**: determine the requested operation and its operands.
- **Execute**: perform the operation, update architectural state, and determine the next PC.
- Real processors may pipeline and overlap these activities; this diagram shows the conceptual instruction cycle, not a particular pipeline.

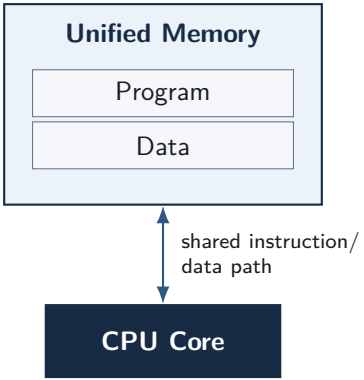
Section 05

von Neumann vs. Harvard Architectures

The von Neumann Architecture

Core Characteristics:

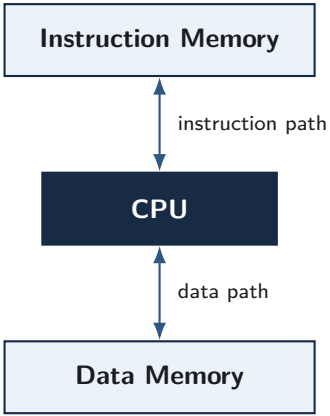
- **Unified Memory**: instructions and data share one memory/address space.
- **Shared Access Path**: instruction fetches and data accesses use the same memory path.
- These accesses can therefore **contend for the same resource**.
- This organization is the classical model associated with the von Neumann bottleneck.



The Harvard Architecture

Core Characteristics:

- **Separated Memories**: instruction and data storage are distinct.
- **Independent Access Paths**: each side has its own path to the processor.
- **Parallel Operation**: instruction fetch and a data read/write can proceed concurrently.
- Instruction and data memories may use different widths or storage technologies.



Shared versus Separate Memory Paths

What Changes?

Instruction access and data access either share one memory path or use separate paths.

Shared Path: von Neumann

- Instruction fetch and data access can compete for the same resource.
- Simpler memory organization and flexible sharing of storage capacity.

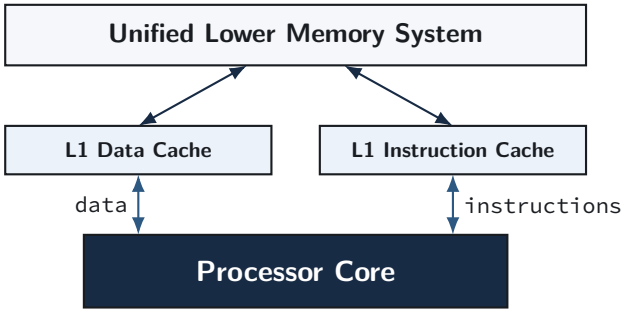
Separate Paths: Harvard

- Removes instruction/data memory-path contention.
- Requires separate storage interfaces and additional hardware resources.

Neither organization eliminates all processor stalls; the distinction concerns the structure of instruction and data access.

Many Modern Processors Use Modified Harvard Organization

Modern high-performance processors often combine elements of both models.



- **Near the core**: separate L1 instruction/data paths permit concurrent fetch and data access.
- **Lower levels**: instructions and data share a unified memory system and address space.
- Examples include Apple M-series and many ARM Cortex-A processors.

Comparison Summary: Memory Organizations

Model	Instruction / Data Storage	Access Structure
von Neumann	Unified instruction and data memory	Shared path; fetch and data access can contend
Harvard	Separate instruction and data memories	Independent paths; concurrent access is possible
Modified Harvard	Separate structures near CPU, unified lower memory	Separate L1 instruction/data paths with shared lower hierarchy

ISA and Memory Organization Are Independent

The ISA does not, by itself, require a processor to use a particular von Neumann, Harvard, or modified-Harvard internal organization.

Section 06

Summary & Self-Test

Topic Summary

- **Architecture vs. Organization**: architecture defines software-visible behavior; organization implements it in hardware.
- **ISA vs. Microarchitecture**: the ISA specifies instructions and architectural state; microarchitecture realizes those semantics with datapaths and control.
- **Stored-Program Concept**: instructions are represented as binary information stored in memory, allowing programs to change without rewiring hardware.
- **von Neumann**: instruction and data accesses share memory resources.
- **Harvard**: instruction and data accesses use separate memories and paths.
- **Modified Harvard**: separate instruction/data structures near the core with a unified lower memory system.

Self-Test: Questions 1–3

Question 1: Architecture vs. Organization

What is the difference between computer architecture and computer organization? Give two examples of each.

Question 2: ISA vs. Microarchitecture

For add `x1, x2, x3`, what does the ISA specify, and what kinds of internal actions belong to the microarchitecture?

Question 3: Stored Program

What is the stored-program concept, and why does it allow one computer to execute many different programs?

Self-Test: Solutions 1–3

Solution 1

Architecture: software-visible specification, such as instruction semantics and visible registers.
Organization: internal implementation, such as pipeline structure and cache organization.

Solution 2

ISA: add the values in `x2` and `x3` and place the result in `x1`.
Microarchitecture: read operands, control the ALU, move the result, and write the destination register.

Solution 3

Instructions are stored as binary information in memory.
Loading a different instruction sequence changes the program without changing the physical hardware.

Self-Test: Questions 4–5

Question 4: von Neumann vs. Harvard

What is the primary structural difference between von Neumann and Harvard organizations? Why can a Harvard organization support simultaneous instruction fetch and data access?

Question 5: Modified Harvard

How does a typical modified-Harvard processor combine separate instruction/data paths near the CPU with a unified lower memory system?

Self-Test: Solutions 4–5

Solution 4

Von Neumann uses a shared instruction/data memory path, so the two access types can contend.
Harvard provides separate instruction and data memories/paths, allowing both accesses to proceed concurrently.

Solution 5

A modified-Harvard processor commonly uses separate L1 instruction and data caches near the core while sharing lower-level caches and main memory within one unified address space.

Next Topic: RISC Principles and ARM Architecture

Topic 2 Preview

- **RISC design principles**
- **AArch64 architectural overview**
- **Programmer-visible registers and state**
- **Major instruction classes**

Topic 2 examines how RISC principles appear in AArch64 and how the ISA presents operations, registers, and architectural state to software.